Ordenação

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br

[1]Faculdade de Sistemas de Informação - UFPA/CUTINS

7 de junho de 2025

Roteiro

Conceitos Gerais
- Objetivos
- Definição
- Terminologia
- Eficiência

Algoritmos Elementares
- Ordenação por Flutuação (Bubble Sort)
- Ordenação por Inserção (Insertion Sort)
- Selection Sort (Ordenação por Seleção)
- Ordenação Rápida (Quick Sort)
- Merge Sort (Ordenação por Intercalação)
- Trabalho 1 - Ordenação

Roteiro

Referências Bibliográficas

Objetivos da Aula

- Esta aula apresenta os conceitos fundamentais de ordenação e os principais algoritmos descritos na literatura.
- Ao final, você deverá ser capaz de:
  - Compreender o processo de comparação e movimentação de elementos em um algoritmo de ordenação.
  - Analisar e comparar o desempenho dos diferentes algoritmos.
  - Identificar aplicações reais onde a ordenação é fundamental.

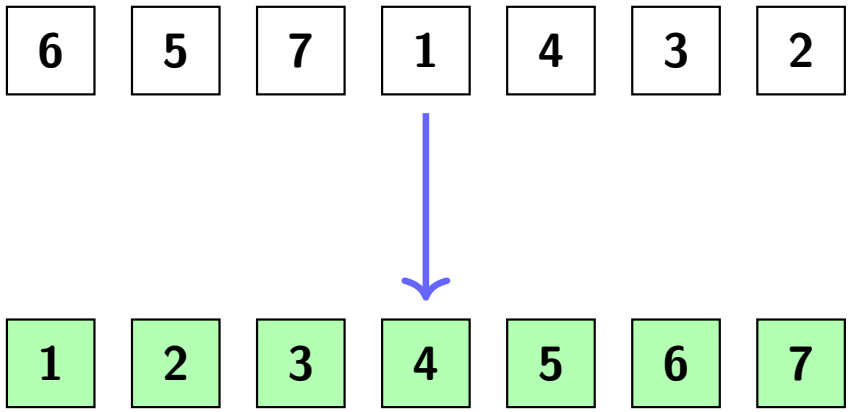
Ordenação, Classificação e Organização

- Um **algoritmo de ordenação** reorganiza uma coleção de itens em uma ordem específica (crescente ou decrescente). A entrada é uma lista, e a saída é uma permutação dessa mesma lista, mas ordenada.
- A organização de dados é um pilar em ciência da computação:
  - Algoritmos de busca (como a busca binária), fusão de listas e outros processos dependem de dados previamente ordenados.
  - Aplicações em bancos de dados, geometria computacional e bioinformática necessitam de listas ordenadas para operar eficientemente.
- Para humanos, dados ordenados são mais fáceis de ler, interpretar e analisar.

Condições de Saída

- A saída de um algoritmo de ordenação deve satisfazer duas condições cruciais:
  1. **Ordem:** A lista final deve estar em uma ordem específica (ex: não decrescente).
  2. **Permutação:** A lista final deve conter exatamente os mesmos elementos da lista original, apenas reorganizados.

Entrada (não ordenada):



Saída (ordenada):

Motivação

- O estudo de algoritmos de ordenação é fundamental para a formação em computação:
  - Solidifica conceitos teóricos importantes:
    - Notação Assintótica (Big O).
    - Análise de pior caso, melhor caso e caso médio.
    - O balanço (trade-off) entre tempo de execução e uso de memória.
  - Como a ordenação é um passo intermediário em muitos algoritmos complexos, é crucial conhecer os métodos mais eficientes para cada cenário.

Terminologia Essencial

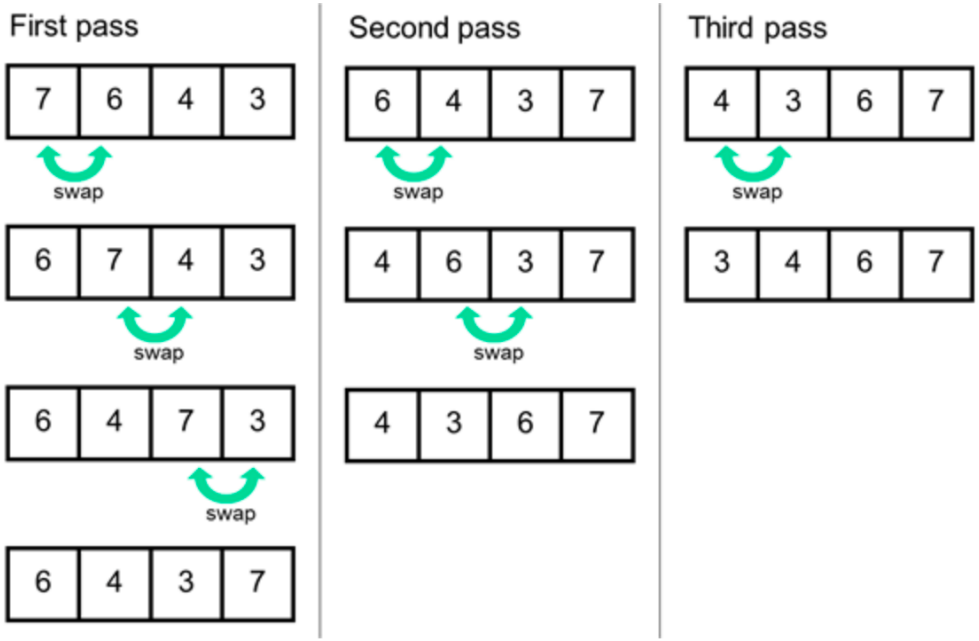
- **Entrada**: Uma sequência de n itens, como $r[0], r[1], ..., r[n-1]$.
- **Registro**: Cada item individual na coleção a ser ordenada.
- **Chave**: Um campo ou valor associado a um registro, sobre o qual a ordenação é realizada. Por exemplo, ordenar uma lista de pessoas pela sua idade.
- **Ordenação interna vs. externa**:
  - **Interna**: Ocorre quando todos os dados cabem na memória principal (RAM).
  - **Externa**: Necessária quando os dados são muito volumosos e precisam ser mantidos em memória secundária (disco), sendo processados em partes.
- **Estabilidade**: Um algoritmo é **estável** se ele preserva a ordem relativa original de registros com chaves iguais.

Medindo a Eficiência

- A eficiência de um algoritmo de ordenação é tipicamente medida através de:
  - **Complexidade de tempo**: Quantas operações o algoritmo executa em função do tamanho da entrada (n).
  - **Complexidade de espaço**: Quanta memória adicional é necessária.
- Para analisar o tempo, contamos as **operações críticas**:
  - **Comparações**: O número de vezes que chaves são comparadas entre si.
  - **Movimentações/Trocas**: O número de vezes que registros são movidos ou trocados de posição.
- O **tamanho da entrada (n)** é o número de registros a serem ordenados.

Bubble Sort: Estratégia

- Compara repetidamente pares de elementos adjacentes.
- Se estiverem na ordem errada, eles são trocados.
- A cada passagem completa, o maior elemento da porção não ordenada "flutua" para sua posição correta no final da lista.



Bubble Sort: Algoritmo

```
funcao BubbleSort(A, tamanho)
  para i de 0 ate tamanho-2 faca
    // A cada passagem, o i-esimo maior elemento
    // eh colocado em sua posicao
    para j de 0 ate tamanho-i-2 faca
      // Compara elementos adjacentes
      se A[j] > A[j+1] entao
        troca(A[j], A[j+1])
      fim se
    fim para
  fim para
fim funcao
```

- **Estabilidade**: Sim, a versão padrão do Bubble Sort é estável.

Bubble Sort: Análise

- O número de comparações é fixo para a versão não otimizada.
- O laço externo roda $n-1$ vezes e o interno também, diminuindo a cada passo.
- **Comparações** (Todos os Casos):

$$C(n) = (n-1) + (n-2) + \cdots + 1 = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

  Portanto, a complexidade é sempre $O(n^2)$ para comparações.
- **Trocas**:
  - **Melhor Caso** (lista ordenada): 0 trocas.
  - **Pior Caso** (ordem reversa): $\frac{n(n-1)}{2}$ trocas.

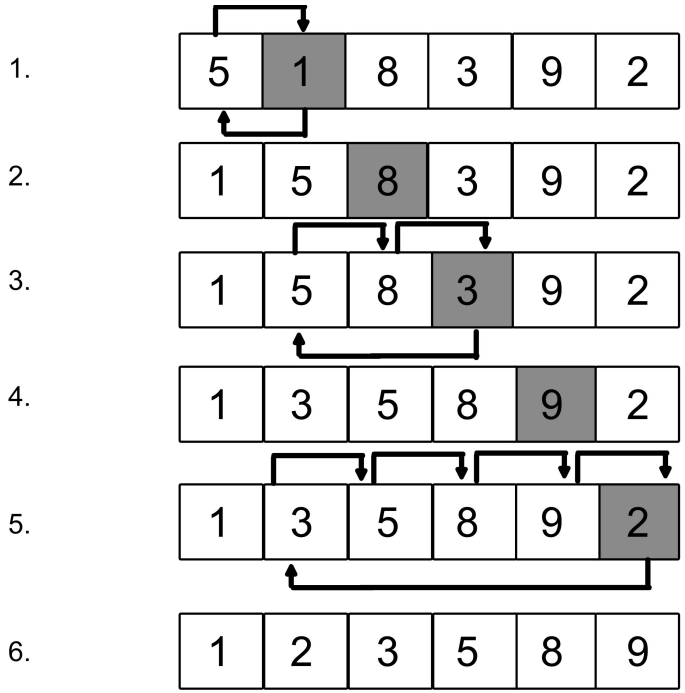
Bubble Sort: Análise

Análise Assintótica

- Mesmo com otimizações, a complexidade geral do Bubble Sort é $O(n^2)$.
- É raramente usado em prática para problemas sérios, mas é um bom algoritmo para fins didáticos.

Insertion Sort: Estratégia

- Percorre-se o vetor da esquerda para a direita. Cada elemento é "inserido" na sua posição correta dentro da sub-lista já ordenada à esquerda.



```
funcao InsertionSort(A, tamanho)
  para j de 1 ate tamanho-1 faca
    chave = A[j] // O elemento a ser inserido
    i = j - 1 // O fim da sub-lista ordenada

    // Move os elementos maiores que a chave
    enquanto (i >= 0) e (A[i] > chave) faca
      A[i+1] = A[i]
      i = i - 1
    fim enquanto

    A[i+1] = chave // Insere a chave
  fim para
fim funcao
```

- **Estabilidade**: Sim, o Insertion Sort é um algoritmo estável.

Insertion Sort: Análise

Análise de Complexidade

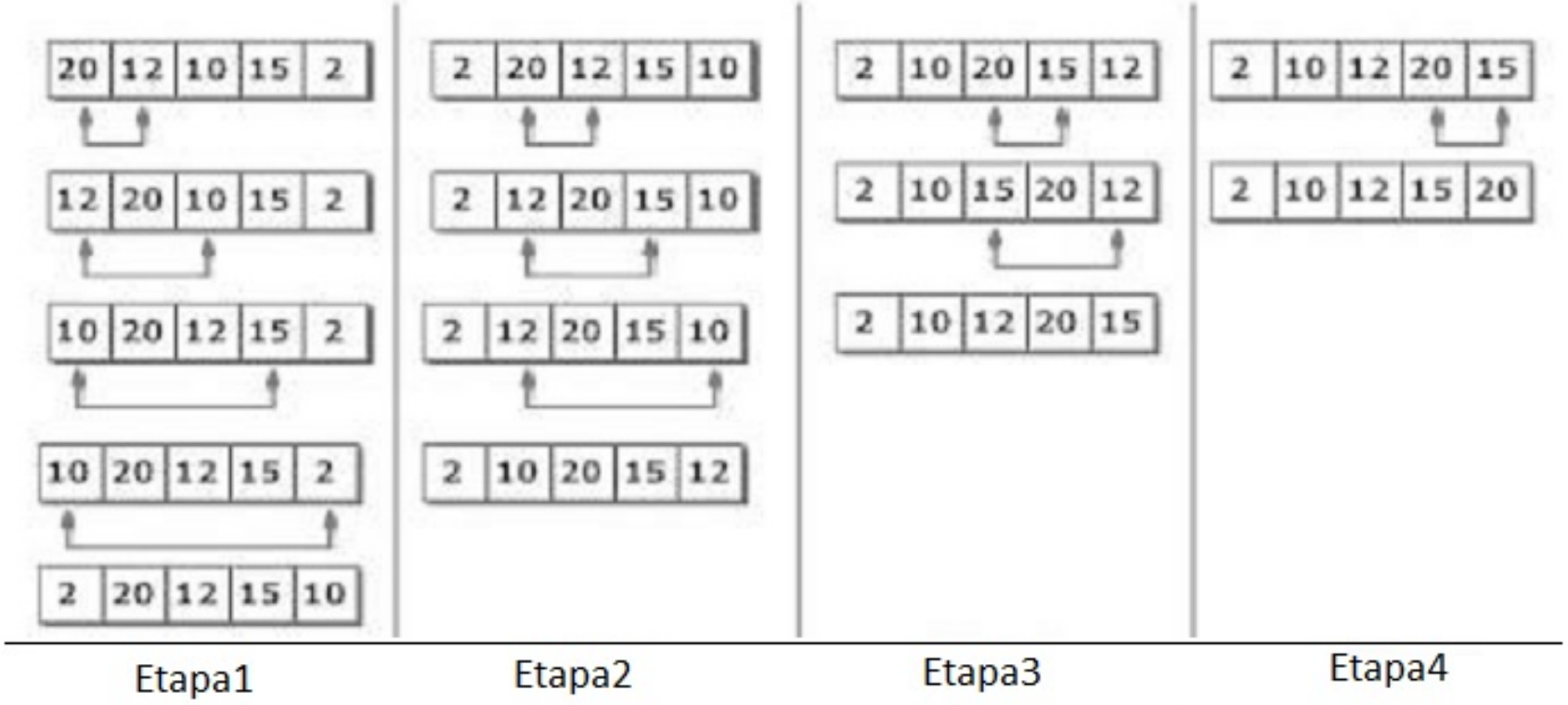
- **Melhor Caso**: A lista já está ordenada.
  - Comparações: O laço 'enquanto' nunca executa seu corpo. $n-1$ comparações. Complexidade: $O(n)$.
  - Movimentações: Nenhuma troca é necessária.
- **Pior Caso**: A lista está em ordem reversa.
  - Comparações: Para cada elemento j, são feitas $j-1$ comparações. Total: $\sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}$. Complexidade: $O(n^2)$.
  - Movimentações: Similar às comparações. Complexidade: $O(n^2)$.
- **Caso Médio**: Complexidade de $O(n^2)$ para comparações e trocas.

- É muito eficiente para listas **pequenas** ou **"quase**

Selection Sort: Estratégia

- Divide a lista em duas partes: uma sub-lista ordenada à esquerda e uma sub-lista não ordenada à direita.
- Em cada iteração, encontra o **menor** elemento na sub-lista não ordenada.
- Troca esse menor elemento com o primeiro elemento da sub-lista não ordenada.
- A fronteira da sub-lista ordenada avança uma posição para a direita.

Estratégia


20 12 10 15 2
12 20 10 15 2
10 20 12 15 2
10 20 12 15 2
2 20 12 15 10
2 20 12 15 10
2 12 20 15 10
2 12 20 15 10
2 10 20 15 12
2 10 20 15 12
2 10 15 20 12
2 10 12 20 15
2 10 12 20 15
2 10 12 15 20
Etapa1
Etapa2
Etapa3
Etapa4

Selection Sort: Algoritmo

```
funcao SelectionSort(A, tamanho)
  para i de 0 ate tamanho-2 faca
    // Encontra o indice do menor elemento restante
    min_idx = i
    para j de i+1 ate tamanho-1 faca
      se A[j] < A[min_idx] entao
        min_idx = j
      fim se
    fim para

    // Troca o menor encontrado com o primeiro
    // elemento da parte nao ordenada
    troca(A[i], A[min_idx])
  fim para
fim funcao
```

Análise de Complexidade

- **Comparações**: O número de comparações é independente da ordem inicial dos dados.
  - Os laços aninhados resultam em $\sum_{i=0}^{n-1}(n-1-i) = \frac{n(n-1)}{2}$ comparações.
  - A complexidade é sempre $\Theta(n^2)$.
- **Trocas**:
  - O número de trocas é no máximo $n-1$, uma por iteração do laço externo.
  - Isso faz do Selection Sort uma boa escolha quando o custo de mover os dados é muito alto.

Análise Assintótica

- O tempo de execução é sempre $\Theta(n^2)$, independentemente da entrada.

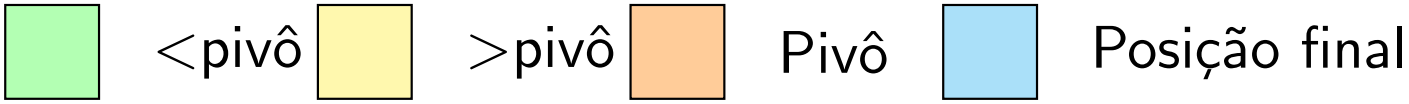
Quick Sort: Características

- Um algoritmo recursivo baseado na estratégia de **dividir para conquistar**.
- Geralmente, é o mais rápido entre os algoritmos de ordenação baseados em comparação para dados em memória.
  - **Caso Médio**: $\Theta(n \log n)$
  - **Pior Caso**: $\Theta(n^2)$ (mas é raro com boas estratégias de pivô)
- O núcleo do método é a rotina de **particionamento**.
  - A partição rearranja a lista em torno de um elemento **pivô**, de modo que todos os elementos à esquerda do pivô são menores, e todos à direita são maiores.
- **Conquistar**: As duas sub-listas (à esquerda e à direita do pivô) são ordenadas recursivamente.
- **Combinar**: Nenhum trabalho é necessário! A ordenação ocorre "no lugar" (in-place).

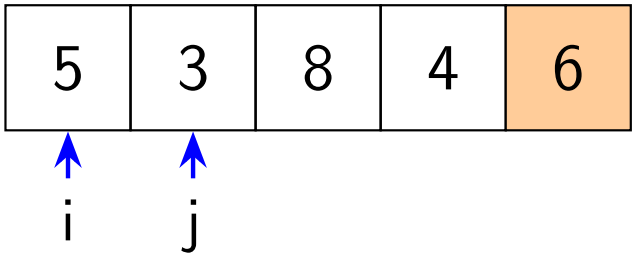
```
funcao QuickSort(A, inicio, fim)
  se inicio < fim entao
    // P recebe o index do pivo que agora esta na posição
        correta
    p = Particao(A, inicio, fim)

    // Ordena recursivamente a sub-lista a esquerda de P
    QuickSort(A, inicio, p - 1)

    // Ordena recursivamente a sub-lista a direita de P
    QuickSort(A, p + 1, fim)
  fim se
fim funcao
```

```
funcao Particao(A, inicio, fim)
  pivo = A[fim]//Estratégia: o ultimo elemento pivo
  // Começa no início do sub-array.
  i = inicio
  // 'j' percorre sub-array p/ encontrar elementos < P.
  para j de inicio ate fim - 1 faca
    // Se um elemento menor que o pivo e encontrado
    se A[j] < pivo entao
      troca(A[i], A[j])//Faz swap
      i = i + 1//avanca p/ proxima posicao.
    fim se
  fim para
  // Ao final,os elementos antes de 'i' sao menores que P.
  // Coloca P em sua posicao final correta, swap com A[i].
  troca(A[i], A[fim])
  // Retorna o index onde P foi colocado.
  retorne i
fim funcao
```

Quicksort - Início da Partição

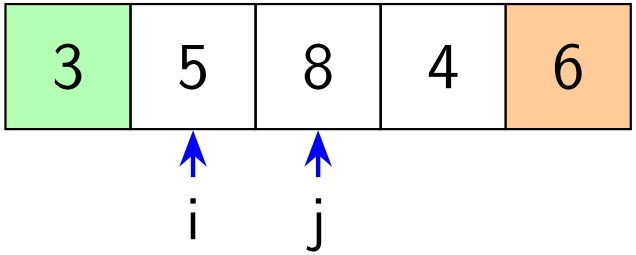


Pivô = 6. Ponteiros i e j iniciam nas posições 0 e 1.



Verifica A[j] < Pivô, caso verdadeiro então faz swap com A[i]

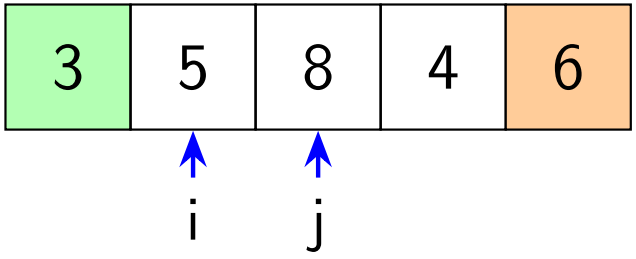
Passo 1: j < pivô



Incrementa i e j.

Verifica A[j] < Pivô, caso verdadeiro então faz swap com A[i]

Passo 1: j < pivô

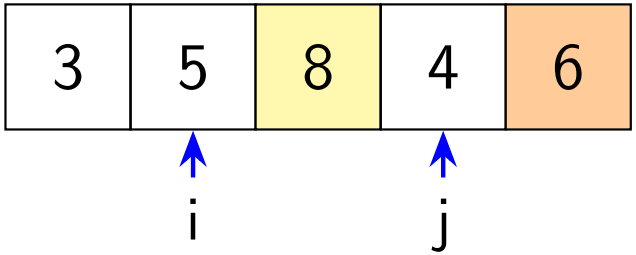


Incrementa i e j.

Verifica A[j] < Pivô, caso verdadeiro então faz swap com A[i]

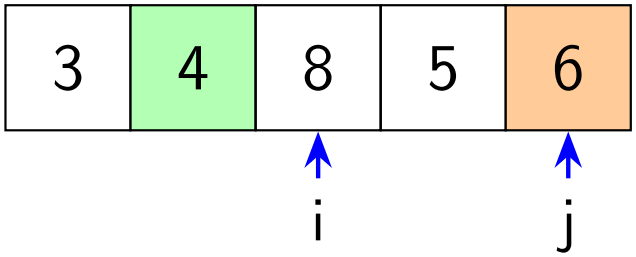
Incrementa j

Passo 2: j > pivô



Verifica A[j] < Pivô, caso verdadeiro então faz swap com A[i]

Passo 3: j < pivô



Incrementa i e j.

Percorreu até final.
Então swap (A[i],A[j])
→ Troca o pivô com o elemento na posição i.)

Passo Final: Troca com o pivô

3	4	6	5	8

Fim da partição.

Quick Sort: Análise

- O desempenho do Quick Sort depende criticamente da **escolha do pivô**.
- O custo do particionamento é sempre linear, $\Theta(n)$.

Pior Caso Ocorre quando o pivô é consistentemente o menor ou o maior elemento. Isso cria partições desbalanceadas (uma com 0 elementos e outra com $n-1$). A complexidade se torna $O(n^2)$.

Melhor Caso O pivô é sempre o elemento mediano, dividindo a lista em duas metades perfeitas. A complexidade é $O(n \log n)$.

Caso Médio Para entradas aleatórias, o desempenho é $O(n \log n)$.

Quick Sort: Análise

Como evitar o pior caso?

- **Pivô Aleatório**: Escolher um elemento aleatório como pivô torna o pior caso extremamente improvável.
- **Mediana de Três**: Escolher o pivô como a mediana do primeiro, meio e último elemento da lista.

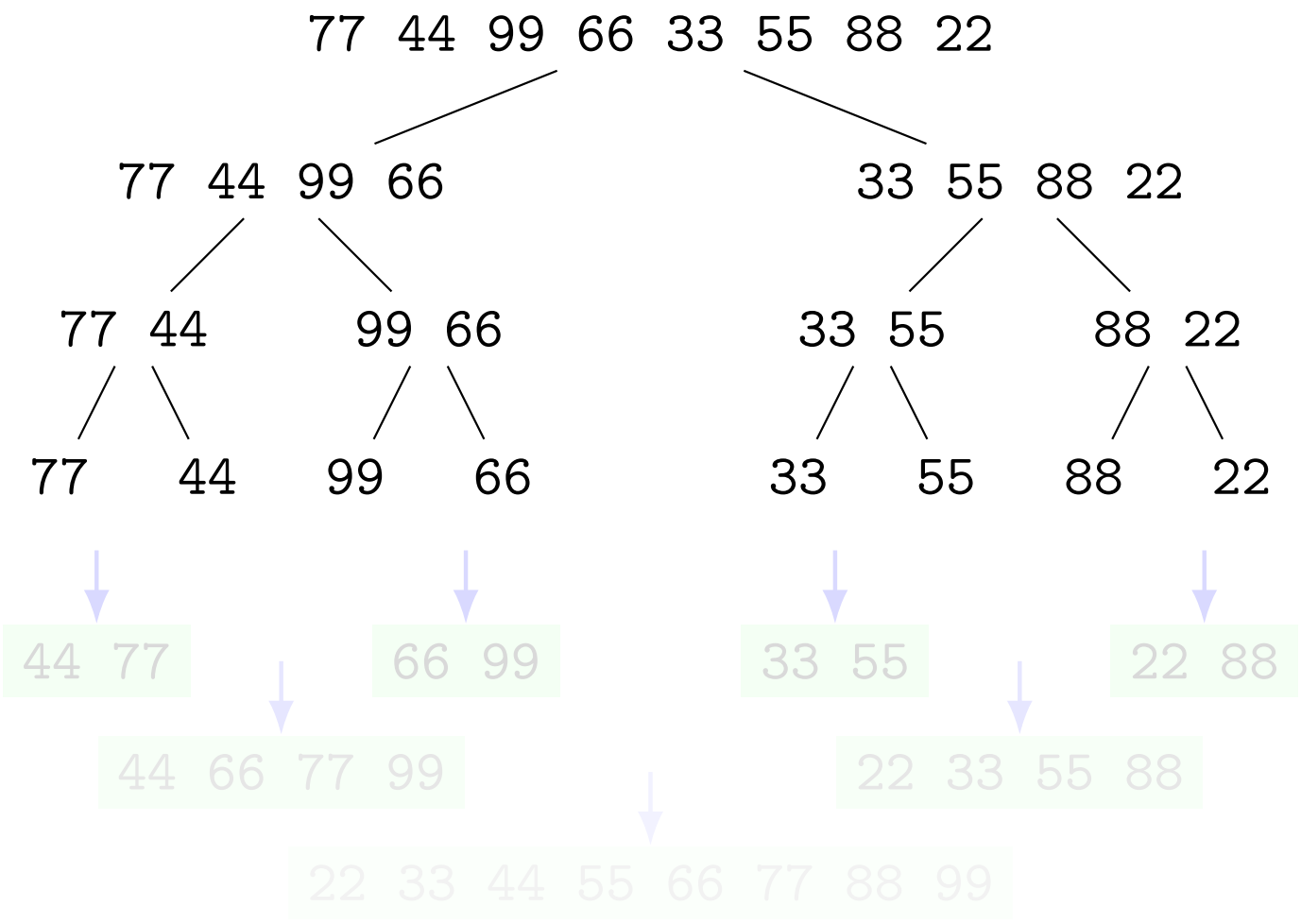
Merge Sort: Estratégia

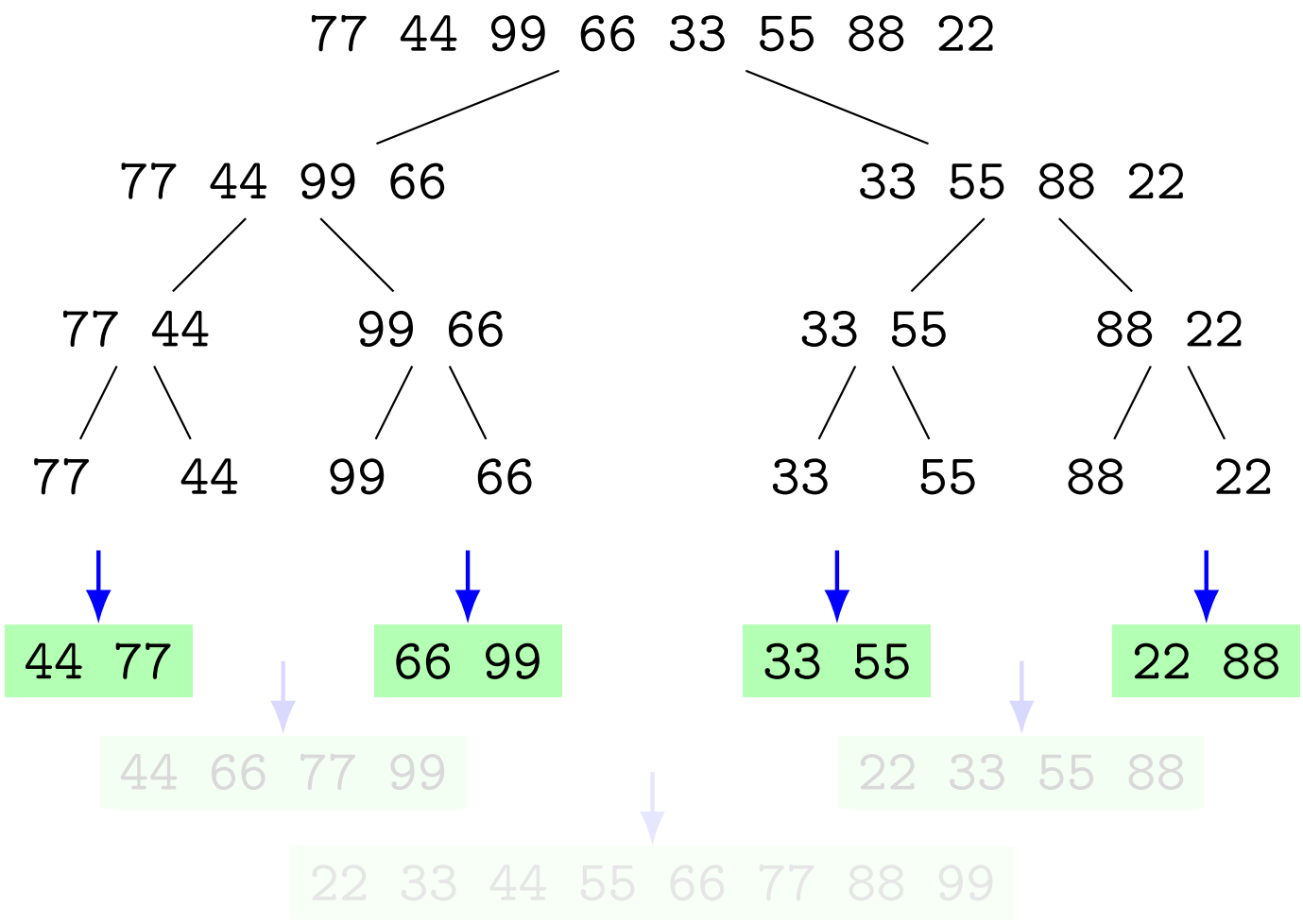
- Também utiliza a abordagem de **dividir para conquistar**.
- O algoritmo segue três passos:

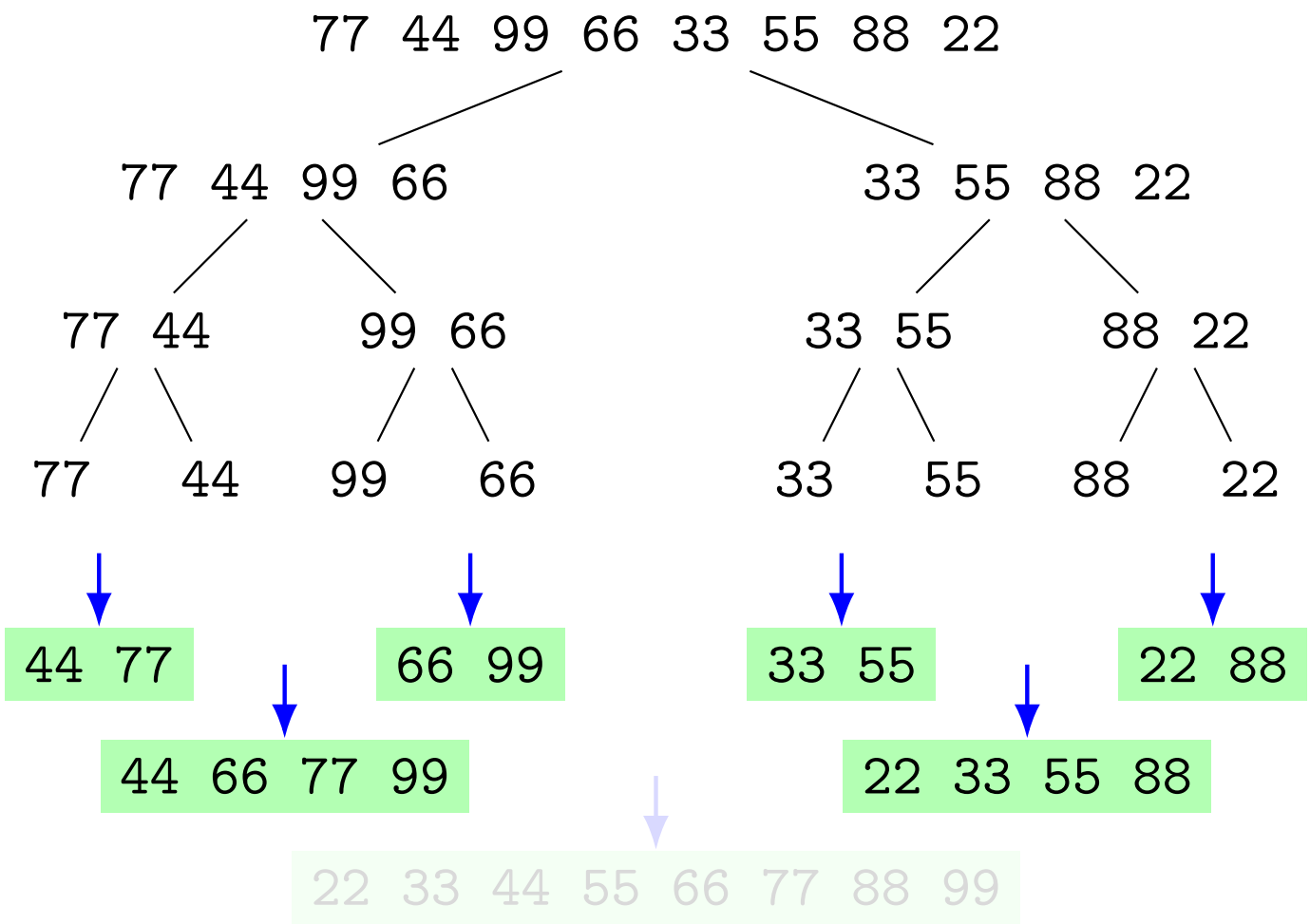
  Dividir O vetor é dividido recursivamente ao meio até que cada sub-vetor contenha apenas um elemento (que, por definição, está ordenado).

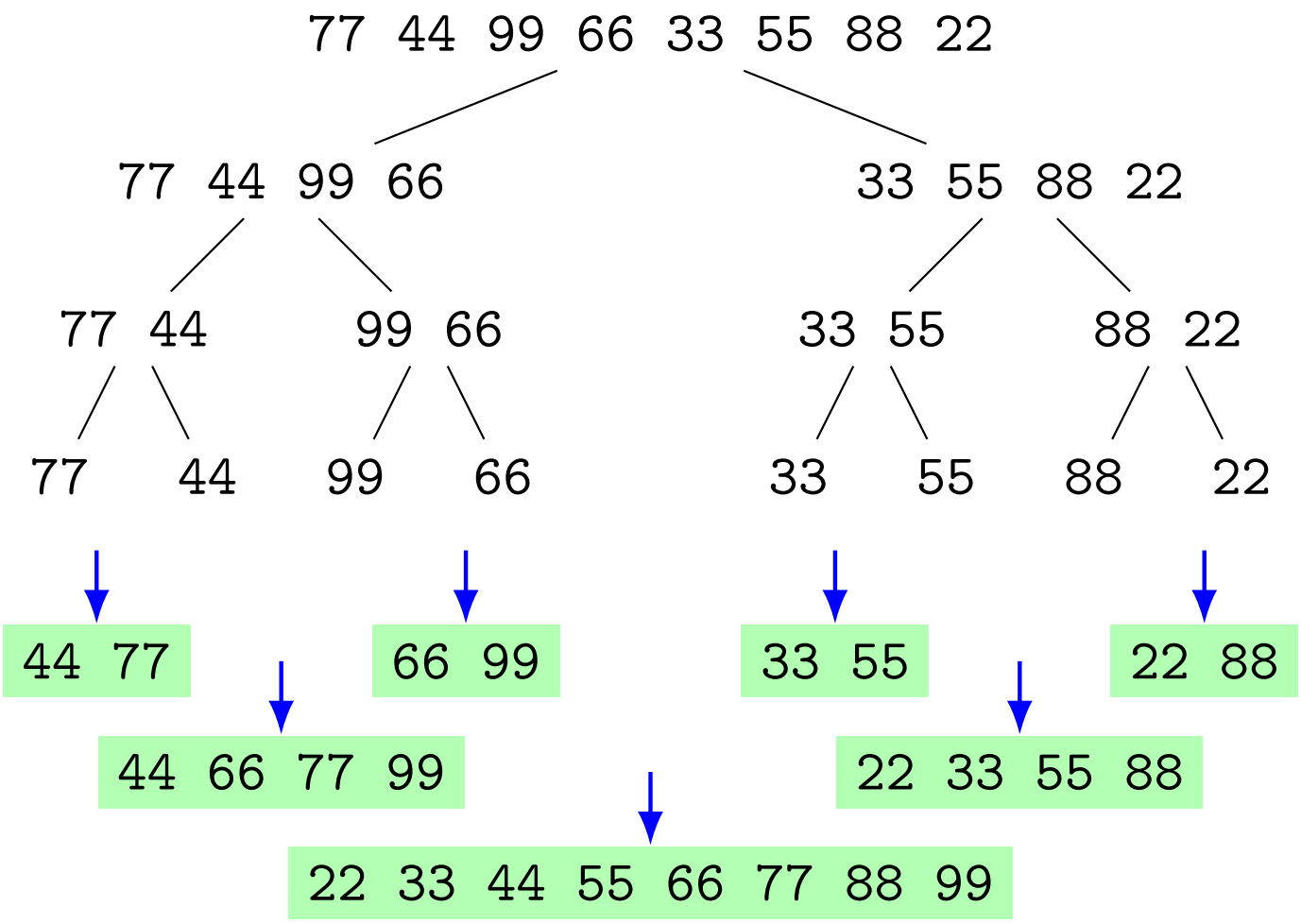
  Conquistar Esta fase é implícita; os vetores de um elemento já estão "conquistados"(ordenados).

  Combinar Os sub-vetores ordenados são **intercalados** (merged) de volta, de dois em dois, para produzir novos sub-vetores ordenados maiores, até que a lista inteira esteja combinada e ordenada.


77 44 99 66 33 55 88 22
77 44 99 66
33 55 88 22
77 44
99 66
33 55
88 22
77
44
99
66
33
55
88
22
44 77
66 99
33 55
22 88
44 66 77 99
22 33 55 88
22 33 44 55 66 77 88 99


77 44 99 66 33 55 88 22
77 44 99 66
33 55 88 22
77 44
99 66
33 55
88 22
77
44
99
66
33
55
88
22
44 77
66 99
33 55
22 88
44 66 77 99
22 33 55 88
22 33 44 55 66 77 88 99


77 44 99 66 33 55 88 22
77 44 99 66
33 55 88 22
77 44
99 66
33 55
88 22
77
44
99
66
33
55
88
22
44 77
66 99
33 55
22 88
44 66 77 99
22 33 55 88
22 33 44 55 66 77 88 99


77 44 99 66 33 55 88 22
77 44 99 66
33 55 88 22
77 44
99 66
33 55
88 22
77
44
99
66
33
55
88
22
44 77
66 99
33 55
22 88
44 66 77 99
22 33 55 88
22 33 44 55 66 77 88 99

Merge Sort: Algoritmo

```
procedimento MergeSort(A, esq, dir)
  se esq < dir entao
    meio = floor((esq + dir) / 2)
    MergeSort(A, esq, meio)
    MergeSort(A, meio + 1, dir)
    Intercalar(A, esq, meio, dir)
  fim se
fim procedimento
```

```
procedimento Intercalar(A, esq, meio, dir)
  // Criar vetores temporarios
  n1 = meio - esq + 1
  n2 = dir - meio
  E[0..n1-1], D[0..n2-1]
  // Copiar dados para os vetores temporarios
  para i de 0 ate n1-1 faca E[i] = A[esq + i]
  para j de 0 ate n2-1 faca D[j] = A[meio + 1 + j]
  // Intercalar de volta para A
  i=0, j=0, k=esq
  enquanto i < n1 e j < n2 faca
    se E[i] <= D[j] entao A[k++] = E[i++]
    senao A[k++] = D[j++]
  fim enquanto
  // Copiar o restante, se houver
  enquanto i < n1 faca A[k++] = E[i++]
  enquanto j < n2 faca A[k++] = D[j++]
fim procedimento
```

Merge Sort: Análise

Análise de Complexidade

- A divisão do vetor em metades cria uma árvore de recursão de altura $\log_2 n$.
- Em cada nível da árvore, a operação de intercalar processa todos os n elementos.
- Portanto, o tempo de execução é o produto da altura da árvore pelo trabalho em cada nível.
- **Tempo de Execução**: $\Theta(n \log n)$ para **todos os casos** (melhor, médio e pior). O comportamento do algoritmo é consistente e não depende da ordem inicial dos dados.

Merge Sort: Análise

Análise de Espaço

- O ponto fraco do Merge Sort é sua necessidade de memória auxiliar.
- A operação de intercalação requer um vetor temporário de tamanho n.
- **Complexidade de Espaço**: $O(n)$.
- **Estabilidade**: Sim, o Merge Sort é um algoritmo estável.

Trabalho Prático

Exercício 1: Comparando Algoritmos Elementares

- Implemente o **Insertion Sort** e o **Bubble Sort**.
- Para cada um, meça o **tempo de execução** e o **número de movimentações/trocas**.
- Execute os experimentos com 3 conjuntos de dados gerados aleatoriamente, com tamanhos:
  - $n = 10.000$
  - $n = 50.000$
  - $n = 100.000$
- Analise e compare os resultados.

Trabalho Prático

Exercício 2: Otimizando o Quick Sort

- Usando como base o algoritmo Quick Sort apresentado, implemente duas estratégias de escolha de pivô: **"mediana-de-três"** e **"versão aleatorizada"**.
- Compare o desempenho (tempo de execução) das novas versões com a implementação original (pivô fixo) para um conjunto de dados aleatórios grande (e.g., $n = 1.000.000$).
- Discuta se as estratégias melhoraram o desempenho e por quê.

Referências I

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms (3rd ed.)*. MIT Press.
- Ascencio, A. F. G., & Araujo, G. S. (2010). *Estrutura de dados: Algoritmos, análise da complexidade e implementações em Java e C++*. Pearson Prentice Hall.
- Szwarcfiter, J. L., & Markenzon, L. (2015). *Estruturas de dados e seus algoritmos (3a ed.)*. LTC.
- Lee, K. D., & Hubbard, S. (2015). Data Structures and Algorithms with Python. Springer.
- Hubbard, J. (2007). *Schaum's Outline of Data Structures with Java*. McGraw-Hill.

Ordenação

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br

[1]Faculdade de Sistemas de Informação - UFPA/CUTINS

7 de junho de 2025